

Основы FastAPI. REST API и архитектура

Введение в FastAPI • обработка и валидация данных • REST API

Что такое FastAPI

FastAPI — это веб-фреймворк для Python, предназначенный для создания **API** (Application Programming Interface — программный интерфейс, через который одна программа взаимодействует с другой). Построен на основе библиотек Starlette и Pydantic и использует обычные подсказки типов Python (type hints) для валидации данных.

Главные преимущества фреймворка:

- **Высокая скорость** — один из самых быстрых Python-фреймворков благодаря минимальным накладным расходам.
- **Простота** — чистый и понятный синтаксис, код пишется быстро.
- **Асинхронность** — умеет эффективно обрабатывать много запросов одновременно.
- **Встроенная валидация данных** — автоматически проверяет параметры запроса и ответы на соответствие ожидаемым типам.
- **Автоматическая документация** — генерирует документацию по стандарту OpenAPI (Swagger, ReDoc) прямо из кода, без ручной работы.

Установка

```
poetry add fastapi[standard]
```

Важно: раньше веб-сервер **Uvicorn** (реализация ASGI — Asynchronous Server Gateway Interface, асинхронного стандарта для Python веб-серверов) нужно было ставить отдельно. Сейчас он устанавливается автоматически вместе с FastAPI.

Первый запуск сервера

Минимальный код приложения в файле `main.py`:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/api/healthchecker")
def root():
    return {"message": "Welcome to FastAPI!"}
```

Что здесь происходит:

- `app = FastAPI()` — создаём экземпляр приложения. Переменная `app` — главная точка входа для всего API.
- `@app.get("/api/healthchecker")` — декоратор, который говорит: функция ниже обрабатывает **GET**-запросы по адресу `/api/healthchecker`.
- Функция возвращает обычный Python-словарь — FastAPI сам превращает его в **JSON** (JavaScript Object Notation — текстовый формат обмена данными) и отправляет клиенту.

HTTP-методы (операции)

При работе с API используются конкретные методы **HTTP** (HyperText Transfer Protocol — протокол передачи данных в вебе). В терминологии OpenAPI и FastAPI их называют **операциями**.

Метод	Декоратор	Действие
GET	@app.get()	Чтение данных
POST	@app.post()	Создание данных
PUT	@app.put()	Полное обновление данных
PATCH	@app.patch()	Частичное обновление данных
DELETE	@app.delete()	Удаление данных

Способы запуска приложения

После установки FastAPI доступна команда `fastapi` в терминале (входит в пакет `fastapi-cli` — Command Line Interface, интерфейс командной строки).

Команда	Назначение	Особенности
<code>fastapi dev</code>	Сервер для разработки	Автоперезагрузка при изменении кода; слушает <code>127.0.0.1</code> (только локальная машина)
<code>fastapi run</code>	Продакшн-сервер	Без автоперезагрузки; слушает <code>0.0.0.0</code> (доступен извне)

```
fastapi dev main.py
```

Более гибкий запуск — напрямую через Uvicorn:

```
uvicorn main:app --host localhost --port 8000 --reload
```

Параметр	Значение
<code>main</code>	Файл <code>main.py</code>
<code>app</code>	Объект приложения (<code>app = FastAPI()</code>)
<code>--host</code>	IP-адрес сервера (по умолчанию <code>127.0.0.1</code>)
<code>--port</code>	Порт сервера (по умолчанию <code>8000</code>)
<code>--reload</code>	Горячая перезагрузка при изменении кода — удобно при разработке

Также приложение можно запустить прямо из Python-скрипта:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/api/healthchecker")
def root():
    return {"message": "Welcome to FastAPI!"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

Автоматическая документация (OpenAPI)

Главное преимущество FastAPI — документация генерируется сама, прямо из кода, без ручной работы.

Адрес	Что показывает
<code>/docs</code>	Интерфейс Swagger UI — можно смотреть и тестировать эндпоинты прямо в браузере
<code>/redoc</code>	Документация в формате ReDoc — более читаемое представление
<code>/openapi.json</code>	«Сырая» схема API в формате OpenAPI (JSON)

Итог: FastAPI сам собирает схему всех маршрутов приложения по стандарту OpenAPI. Раньше такую документацию пришлось бы писать вручную в файле JSON или YAML — теперь она формируется автоматически из кода.

Получение и валидация данных

Данные от клиента на сервер можно получить несколькими способами: через параметры пути, параметры запроса, тело запроса или заголовки. Разберём каждый — и как FastAPI валидирует данные на каждом из них.

Параметры пути (Path)

Path-параметр — часть URL (Uniform Resource Locator — адрес ресурса в интернете), которая помечается фигурными скобками `{}`. Используется для получения динамических значений — например, id конкретной записи.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/notes/{note_id}")
async def read_note(note_id: int):
    return {"note": note_id}
```

Запрос `/notes/10` вернёт `{"note": 10}`. Раз мы указали тип `note_id: int`, валидация подключается автоматически: запрос `/notes/foo` вернёт ошибку — Pydantic проверит, что значение действительно целое число, без единой дополнительной строчки кода.

Частая ошибка — порядок маршрутов. FastAPI проверяет маршруты последовательно сверху вниз. Если маршрут `/notes/{note_id}` объявлен раньше, чем `/notes/new`, то запрос на `/notes/new` попадёт в первый обработчик — FastAPI попытается превратить `"new"` в число и выдаст ошибку валидации. Поэтому **конкретные маршруты всегда объявляются раньше общих** с параметром пути:

```
@app.get("/notes/new")
async def read_new_notes():
    return {"message": "Return new notes"}

@app.get("/notes/{note_id}")
async def read_note(note_id: int):
    return {"note": note_id}
```

Дополнительные проверки для параметра пути задаются классом `Path`:

```
from fastapi import FastAPI, Path

@app.get("/notes/{note_id}")
async def read_note(note_id: int = Path(description="The ID of the note to get", gt=0, le=10)):
    return {"note": note_id}
```

Параметр	Проверка
gt	значение должно быть больше указанного
ge	больше или равно
lt	меньше указанного
le	меньше или равно
min_length	минимальная длина строки
max_length	максимальная длина строки
regex	значение должно соответствовать регулярному выражению

Параметры запроса (Query)

Query-параметры — пары «ключ=значение» в конце URL после знака `?`, разделённые символом `&`. Используются для фильтров, пагинации, сортировки: `/notes/?skip=0&limit=10`.

```
@app.get("/notes")
async def read_notes(skip: int = 0, limit: int = 10):
    return {"message": f"Return all notes: skip: {skip}, limit: {limit}"}
```

Любой параметр функции-обработчика, который **не встречается** в пути маршрута, FastAPI автоматически считает query-параметром.

Чтобы параметр стал необязательным, ему нужно значение по умолчанию — например, `None`. Без значения по умолчанию параметр становится обязательным. Дополнительная валидация задаётся классом `Query` (параметры те же, что и у `Path`):

```
@app.get("/notes")
async def read_notes(skip: int = 0, limit: int = Query(default=10, le=100, ge=10)):
    return {"message": f"Return all notes: skip: {skip}, limit: {limit}"}
```

Тело запроса (Body)

Тело запроса — данные, которые передаются вместе с HTTP-запросом (в формате JSON, XML — eXtensible Markup Language, «расширяемый язык разметки» — или другом). Используется в запросах `POST`, `PUT`, `PATCH` для создания или изменения данных. Формат тела указывается заголовком `Content-Type` (например, `application/json`).

Схема тела запроса описывается через модель Pydantic:

```
from fastapi import FastAPI
from pydantic import BaseModel

class Note(BaseModel):
    name: str
    description: str
    done: bool

@app.post("/notes")
async def create_note(note: Note):
    return {"name": note.name, "description": note.description, "status": note.done}
```

FastAPI сам разбирает JSON из тела запроса, проверяет соответствие полей типам `Note` и передаёт готовый объект в функцию. Если данные не соответствуют схеме — вернётся ошибка валидации, без единой ручной проверки.

Проще всего протестировать это прямо в Swagger UI (`/docs`): открываете маршрут, нажимаете «Try it out», вставляете JSON и получаете реальный ответ сервера — документация сама выступает клиентом для API.

Заголовки (Headers)

HTTP-заголовки — дополнительная метаданная при запросе или ответе: тип контента, токены авторизации, данные о кэшировании и т.д. Для доступа к ним используется класс `Header` :

```
from fastapi import FastAPI, Header

@app.get("/headers")
async def read_headers(user_agent: str = Header(default=None)):
    return {"User-Agent": user_agent}
```

FastAPI автоматически превращает названия заголовков из `kebab-case` в `snake_case` : заголовок `user-agent` становится параметром `user_agent` .

Чтобы получить сразу все заголовки — используется объект `Request` :

```
from fastapi import FastAPI, Request

@app.get("/all-headers")
async def read_all_headers(request: Request):
    return {"headers": dict(request.headers)}
```

Как и с `Path` / `Query` , для заголовков можно задать валидацию (например, `min_length` , `max_length`). Если данные не проходят проверку, FastAPI сам вернёт ответ со статусом `422 Unprocessable Entity` и подробным описанием ошибки в JSON.

Расширенная валидация: Pydantic

Pydantic снимает большинство вопросов валидации входных данных и убирает человеческий фактор. Поля модели можно ограничивать через `Field` , а для распространённых форматов есть готовые типы — например, `EmailStr` (проверенный email) и `HttpUrl` (проверенный URL):

```
from typing import Optional
from pydantic import BaseModel, Field, EmailStr, HttpUrl

class User(BaseModel):
    name: str
    email: EmailStr
    website: HttpUrl
    age: Optional[int] = Field(None, ge=13, le=90)

# Некорректный возраст (меньше 13) — вызовет ValidationError
user = User(name="Jane", email="jane@example.com", website="https://jane.com", age=12)
```

Если значение поля не проходит проверку — Pydantic выбрасывает исключение `ValidationError` с точным указанием, какое поле и почему не прошло валидацию.

Важно понимать границу: Pydantic проверяет только тип и формат данных (число ли это, похож ли текст на email или URL). Бизнес-логику — например, «такой email уже зарегистрирован в системе» — нужно проверять отдельной функцией, Pydantic это не делает.

Подключение к базе данных

FastAPI можно подключить к любой реляционной базе данных. Здесь используется **SQLAlchemy** (ORM — Object-Relational Mapping, «объектно-реляционное отображение»: позволяет работать с таблицами БД как с обычными Python-классами) и **PostgreSQL**.

Настройка SQLAlchemy

Запускаем PostgreSQL в Docker-контейнере:

```
docker run --name db-postgres -p 5432:5432 -e POSTGRES_PASSWORD=567234 -d postgres
```

Ставим зависимости — саму SQLAlchemy и драйвер `psycopg2` для PostgreSQL:

```
poetry add sqlalchemy psycopg2
```

Файл `db.py` — подключение и модель таблицы:

```
from sqlalchemy import create_engine, Integer, String, Boolean
from sqlalchemy.orm import sessionmaker, DeclarativeBase, mapped_column, Mapped

DATABASE_URL = "postgresql+psycopg2://postgres:567234@localhost:5432/postgres"
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

class Base(DeclarativeBase):
    pass

class Note(Base):
    __tablename__ = "notes"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(50))
    description: Mapped[str] = mapped_column(String(250))
    done: Mapped[bool] = mapped_column(Boolean, default=False)

Base.metadata.create_all(bind=engine)

# Dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Что здесь важно: `engine` — подключение к БД по адресу `DATABASE_URL`. `SessionLocal` — фабрика сессий для запросов. `Base` — базовый класс, от которого наследуются модели таблиц; класс `Note` описывает таблицу `notes`. Функция `get_db` — это **зависимость** (dependency): открывает сессию, отдаёт её через `yield` и гарантированно закрывает после использования, что бы ни случилось внутри запроса.

Что такое Depends. Запись `db: Session = Depends(get_db)` говорит FastAPI: «это значение приходит извне». Перед вызовом функции-обработчика FastAPI сам вызовет `get_db()`, получит сессию и подставит её в аргумент `db`. Это и есть механизм **Dependency Injection** (внедрение зависимостей) в FastAPI — переиспользуемая подготовка данных для обработчиков.

Работа с сессией в обработчиках

Проверка подключения к базе прямо в маршруте:

```

from fastapi import FastAPI, Depends, HTTPException, status
from sqlalchemy import text
from sqlalchemy.orm import Session
from db import get_db

@app.get("/api/healthchecker")
def healthchecker(db: Session = Depends(get_db)):
    try:
        result = db.execute(text("SELECT 1")).fetchone()
        if result is None:
            raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail="Database is not")
        return {"message": "Welcome to FastAPI!"}
    except Exception as e:
        raise HTTPException(status_code=status.HTTP_500_INTERNAL_SERVER_ERROR, detail="Error connecting to

```

А сохранение записи в базе — вместо простого возврата словаря, как было раньше:

```

class NoteModel(BaseModel):
    name: str
    description: str
    done: bool

@app.post("/notes")
async def create_note(note: NoteModel, db: Session = Depends(get_db)):
    new_note = Note(name=note.name, description=note.description, done=note.done)
    db.add(new_note)
    db.commit()
    db.refresh(new_note)
    return new_note

```

`db.add()` добавляет запись в сессию, `db.commit()` сохраняет её в базе, `db.refresh()` подтягивает из базы актуальные данные записи (например, сгенерированный `id`) — и функция возвращает уже сохранённый объект.

Здесь показано **синхронное** подключение — для понимания самого принципа. В реальных проектах на FastAPI обычно используют асинхронное подключение к базе, чтобы не терять преимущество асинхронности фреймворка.

Схема ответа (Response Model)

Без явного указания схемы, документация Swagger не знает, что именно возвращает обработчик — вместо реальной структуры показывает просто «string». Чтобы это исправить, схему ответа описывают точно так же, как схему входных данных — через Pydantic-модель.

```

class ResponseNoteModel(BaseModel):
    id: int = Field(default=1, ge=1)
    name: str
    description: str
    done: bool

```

Указать её можно двумя способами. Первый — аннотацией возвращаемого типа функции:

```
@app.get("/notes")
async def read_notes(
    skip: int = 0,
    limit: int = Query(default=10, le=100, ge=10),
    db: Session = Depends(get_db),
) → list[ResponseNoteModel]:
    return db.query(Note).offset(skip).limit(limit).all()
```

Второй — параметром `response_model` в декораторе маршрута:

```
@app.get("/notes/{note_id}", response_model=ResponseNoteModel)
async def read_note(
    note_id: int = Path(description="The ID of the note to get", gt=0, le=10),
    db: Session = Depends(get_db),
):
    note = db.query(Note).filter(Note.id == note_id).first()
    if note is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Not found")
    return note
```

`status` — объект из `fastapi` с готовыми константами HTTP-кодов (`HTTP_404_NOT_FOUND`, `HTTP_200_OK`, `HTTP_201_CREATED` и другие) — с ними код читается понятнее, чем с «голыми» числами. `HTTPException` — встроенное исключение FastAPI для ответа об ошибке с нужным статус-кодом.

Оба способа рабочие — выбор обычно дело вкуса и стиля команды. Дальше в конспекте используется вариант с `response_model`: он явнее показывает схему ответа прямо в декораторе маршрута. Главная польза схемы ответа — ошибки ловятся раньше, до того как попадут к клиенту, а формат ответа сразу задокументирован.

Обработка ошибок

Базовый способ вернуть ошибку — «выбросить» исключение `HTTPException` с нужным статус-кодом:

```
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    item = get_item(item_id)
    if item is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Item not found")
    return item
```

400 или 422?

FastAPI сам возвращает `422 Unprocessable Entity` на любую ошибку валидации Pydantic. Но при ручных проверках нужно выбирать код осознанно:

Код	Когда использовать
400 Bad Request	Неправильный формат запроса или отсутствуют обязательные данные — например, вместо JSON пришёл plain text, или число передано строкой.
422 Unprocessable Entity	Формат запроса верный, но данные не проходят по смыслу или бизнес-логике — например, дата окончания раньше даты начала, или email отформатирован правильно, но недействителен.

Единый обработчик ошибок

Декоратор `@app.exception_handler()` перехватывает исключения определённого типа во всём приложении — удобно, чтобы привести формат ошибок к единому виду:

```

from fastapi.responses import JSONResponse

@app.exception_handler(HTTPException)
def http_exception_handler(request: Request, exc: HTTPException):
    return JSONResponse(
        status_code=exc.status_code,
        content={"message": exc.detail},
    )

```

Здесь везде, где раньше в ответе было поле `"detail"`, теперь будет `"message"` — единый формат ошибок для всего API. Поскольку формат ответа изменился, Swagger нужно явно сообщить новую схему ошибки через параметр `responses` декоратора:

```

class ErrorResponse(BaseModel):
    message: str

@app.post("/items/", responses={400: {"model": ErrorResponse}})
async def create_item(item: Item):
    return item

```

Кастомная валидация полей модели делается через `@model_validator(mode="before")` из Pydantic — эта проверка срабатывает до того, как Pydantic начнёт свою стандартную проверку типов, и внутри неё тоже можно поднимать `HTTPException` с нужным статусом и сообщением.

Для доменных ошибок удобно заводить собственные классы исключений вместо переиспользования `HTTPException`:

```

class ItemNotFoundError(Exception):
    pass

@app.exception_handler(ItemNotFoundError)
def item_not_found_error_handler(request: Request, exc: ItemNotFoundError):
    return JSONResponse(status_code=status.HTTP_404_NOT_FOUND, content={"message": "Item not found"})

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    item = get_item_by_id(item_id)
    if item is None:
        raise ItemNotFoundError
    return item

```

Итор: `@app.exception_handler()` позволяет централизованно управлять форматом ошибок и статус-кодами по всему приложению — вместо того чтобы формировать ответ об ошибке вручную в каждом обработчике.

Промежуточное ПО (Middleware)

Middleware — компонент, который стоит между клиентом и сервером и обрабатывает каждый запрос и ответ, проходящие через приложение. Позволяет добавлять функциональность (авторизацию, логирование, лимиты запросов), не трогая код самих обработчиков.

Как это работает:

1. Клиент отправляет запрос на endpoint FastAPI-приложения.
2. Запрос сначала попадает в middleware — раньше, чем в обработчик маршрута.
3. Middleware может сразу прервать обработку и вернуть свой ответ клиенту, не доходя до обработчика — например, вернуть `401 Unauthorized`, если запрос не прошёл аутентификацию, или `429 Too Many Requests` при

превышении лимита запросов.

4. Если middleware не прерывает поток — запрос идёт дальше, к обработчику маршрута.
5. Ответ обработчика проходит через middleware ещё раз, уже в обратном направлении, перед отправкой клиенту.

```
from fastapi import FastAPI, Request
import time

app = FastAPI()

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response
```

Эта функция работает для каждого запроса. `call_next(request)` — вызов следующего звена цепочки (другого middleware или, если их больше нет, самого обработчика маршрута). Функция засекает время до и после вызова `call_next`, а разницу добавляет в заголовок ответа `X-Process-Time` — удобно для мониторинга производительности.

Прерывание запроса. Чтобы middleware завершил обработку сам, не дожидаясь обработчика, — достаточно просто не вызывать `call_next`, а сразу вернуть готовый ответ:

```
from fastapi.responses import JSONResponse

async def example_middleware(request: Request, call_next):
    if some_condition:
        return JSONResponse(content={"message": "Request intercepted"}, status_code=400)
    response = await call_next(request)
    return response
```

Статические ресурсы

Статические ресурсы — файлы, которые сервер отдаёт клиенту без изменений: HTML, CSS, JavaScript, изображения, видео. В отличие от динамического контента, они не генерируются заново на каждый запрос.

Класс `StaticFiles` монтирует директорию на диске как маршрут:

```
from fastapi import FastAPI
from fastapi.staticfiles import StaticFiles

app = FastAPI()
app.mount("/static", StaticFiles(directory="static"), name="static")
```

`/static` — URL-путь, по которому будут доступны файлы; `directory="static"` — папка на диске; `name="static"` — имя монтирования, нужно для обратной генерации URL. После этого файл `static/css/style.css` становится доступен по адресу `/static/css/style.css`. Префикс URL можно менять, и можно монтировать сразу несколько директорий (например, отдельно `/static` и `/media`).

Типичная структура проекта:

```
myproject/
├─ main.py
├─ static/
│   ├─ css/main.css
│   ├─ js/script.js
│   └─ images/logo.png
└─ templates/
    └─ index.html
```

Безопасность. По умолчанию FastAPI не отдаёт HTML-файлы из статической директории напрямую (запрос вернёт 404) — это защита от случайной утечки файлов. Параметр `html=True` в `StaticFiles(directory="static", html=True)` снимает это ограничение, но включать его стоит, только если вы полностью контролируете содержимое директории.

Отдать HTML можно двумя способами. Через шаблонизатор Jinja2:

```
from fastapi.templating import Jinja2Templates

templates = Jinja2Templates(directory="templates")

@app.get("/")
async def read_root(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})
```

Или напрямую через `HTMLResponse`, без шаблонизатора:

```
from fastapi.responses import HTMLResponse

@app.get("/", response_class=HTMLResponse)
async def read_root():
    with open("static/index.html") as f:
        return f.read()
```

Параметр `name="static"` из `app.mount()` позволяет генерировать URL через `request.url_for("static", path="images/logo.png")` — если позже поменяете префикс монтирования, весь код с `url_for` продолжит работать без правок.

Работа с формами и файлами

Загрузка файлов

Нужна библиотека `python-multipart` (в свежих версиях ставится вместе с самим FastAPI):

```
poetry add python-multipart
```

```

from fastapi import FastAPI, File, UploadFile
import pathlib

app = FastAPI()

@app.post("/uploadfile/")
async def create_upload_file(file: UploadFile = File()):
    pathlib.Path("uploads").mkdir(exist_ok=True)
    file_path = f"uploads/{file.filename}"
    with open(file_path, "wb") as f:
        f.write(await file.read())
    return {"file_path": file_path}

```

`UploadFile` получает загруженный файл из запроса; код создаёт папку `uploads` (если её ещё нет) и сохраняет туда файл под тем же именем.

На заметку: для реального проекта с большим объёмом или крупными файлами стоит хранить их не на диске сервера, а в специализированном хранилище — AWS S3, Google Cloud Storage, Cloudinary. Также важно проверять права доступа и не позволять загрузку/выгрузку файлов из корневой директории проекта — это риск утечки чувствительных данных.

Загрузка файла из HTML-формы

В форме обязательно указывается `enctype="multipart/form-data"` — без этого атрибута файлы не отправятся:

```

<form action="/" enctype="multipart/form-data" method="post">
  <input type="file" name="file">
  <input type="submit" value="Submit">
</form>

```

Обработчик на стороне сервера принимает файл так же, как в примере выше, только теперь ещё и отдаёт саму форму на GET-запрос:

```

from fastapi import FastAPI, File, UploadFile, Request
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
import pathlib

app = FastAPI()
UPLOAD_DIR = pathlib.Path(__file__).parent / "uploads"
UPLOAD_DIR.mkdir(exist_ok=True)

templates = Jinja2Templates(directory=".")
app.mount("/static", StaticFiles(directory=UPLOAD_DIR), name="static")

@app.get("/")
async def index(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@app.post("/")
async def create_upload_file(request: Request, file: UploadFile = File()):
    file_path = UPLOAD_DIR / file.filename
    with open(file_path, "wb") as f:
        f.write(await file.read())
    return {"file_path": request.url_for("static", path=file.filename)}

```

После отправки формы загруженный файл сразу доступен в браузере по смонтированному статическому маршруту — например, как изображение.

Основы API

API (Application Programming Interface — программный интерфейс) — описание способов, которыми одна программа может взаимодействовать с другой, не зная деталей её внутренней реализации. API есть у операционных систем, библиотек и, конечно, у веб-сервисов — тогда его называют **Web API**.

Самый распространённый стандарт для Web API сегодня — **REST**. Также встречаются **GraphQL** (растущая популярность), **SOAP** и более старый **XML-RPC** (Remote Procedure Call — протокол вызова удалённых процедур поверх XML, используется в основном ради обратной совместимости в старых системах).

JSON API — надстройка над REST с дополнительными требованиями: тело ответа должно существовать всегда (если возвращать нечего — используют статус без тела, например `204`), а сам ответ обязан содержать хотя бы одно из полей: `data` (запрошенные данные), `errors` (список ошибок) или `meta` (всё остальное). На практике из этого стандарта чаще всего берут только сам формат данных.

Подход API First

Современный веб-сервис обычно состоит из Front-end (отображение данных) и Back-end (хранение и обработка данных), которые часто делают разные команды. Если Back-end сначала пишет код и только потом показывает готовый API — Front-end простаивает, а ошибки проектирования API вскрываются слишком поздно, уже перед релизом.

API First переворачивает порядок: сначала команды вместе согласовывают спецификацию API, и только потом обе стороны начинают писать код — Front-end на основе **mock-объектов** (заглушек, имитирующих реальный API), Back-end — реализуя согласованный контракт.

Ключевые преимущества подхода:

- Front-end и Back-end работают параллельно, а не по очереди.
- Ошибки проектирования находятся на этапе спецификации — это дешевле, чем исправлять их в готовом коде.
- Спецификация API — единый источник правды для всех команд, меньше недопонимания.
- Систему проще масштабировать и менять по частям, не ломая остальное.
- Документация актуальна всегда, потому что она и есть основа разработки, а не написана «постфактум».

Чтобы подход работал, нужен стандарт описания API — самый распространённый из них **Swagger (OpenAPI)**: язык описания API на основе разметки **YAML** (Yet Another Markup Language — «человекочитаемый» формат сериализации данных), из которого можно сгенерировать документацию, серверный и клиентский код:

```
swagger: '2.0'
info:
  title: Swagger REST Article
basePath: '/api'
paths:
  /people:
    get:
      summary: 'Список людей'
      responses:
        200:
          description: 'Успешный ответ'
```

FastAPI генерирует такую документацию автоматически из кода — отдельно писать YAML-спецификацию, как в примере выше, не требуется.

Что такое REST

REST (REpresentational State Transfer — передача репрезентативного состояния) — не протокол и не стандарт, а набор архитектурных принципов для масштабируемых и гибких сетевых систем. Термин ввёл Рой Филдинг в 2000 году. Система, соответствующая этим принципам, называется **RESTful**.

Перед разбором принципов — важная разница между двумя похожими терминами:

Термин	Значение
URI	Идентификатор конкретного ресурса (как название книги)
URL	Идентификатор, который ещё и говорит, как получить доступ к ресурсу (как адрес книги в библиотеке)

Структура URL:

```
<схема>: // [<логин>[:<пароль>]@]<хост>[:<порт>] [/<путь>] [ ?<запрос>] [#<фрагмент>]
```

Часть	Значение
схема (scheme)	протокол доступа к ресурсу (http , https)
хост (host)	доменное имя или IP-адрес сервера
порт (port)	порт подключения (по умолчанию 80 для HTTP, 443 для HTTPS — поэтому в браузере его обычно не видно)
путь (path)	расположение ресурса на сервере
запрос (query)	параметры после ? , разделённые &
фрагмент (fragment)	якорь после # — например, ссылка на секцию страницы

6 архитектурных ограничений RESTful

Ограничение	Суть
Клиент-сервер	Роли строго разделены: сервер хранит и обрабатывает ресурсы, клиент их запрашивает. Это позволяет развивать обе стороны независимо.
Stateless (без состояния)	Сервер не хранит контекст между запросами — каждый запрос содержит всю информацию, нужную для его обработки, без «сессий» на сервере.
Единый интерфейс	Стандартный, предсказуемый способ общения клиента и сервера — раскрыт подробнее ниже.
Кэширование	Ответ сервера помечается как кэшируемый или нет — это снижает число повторных запросов и повышает производительность.
Многослойность системы	Между клиентом и сервером могут быть промежуточные слои — прокси, шлюзы — при этом каждый компонент «видит» только соседний слой.
Код по требованию (необязательное)	Сервер может передать клиенту исполняемый код — например, так браузер получает и выполняет JavaScript.

Единый интерфейс — самое ёмкое из ограничений, оно состоит из четырёх частей:

- **Идентификация ресурсов** — у каждого ресурса стабильный идентификатор (URL), который не меняется при взаимодействии с ним.
- **Управление ресурсом через представление** — клиент не меняет ресурс напрямую, а отправляет серверу его представление (обычно JSON) с желаемым результатом; решение и контроль остаются за сервером.
- **Самоописательные сообщения** — сообщение содержит всю информацию для его понимания (например, заголовок Content-Type говорит, как читать тело ответа) — не нужно лезть в отдельную документацию.
- **Гипермедиа** — ответ сервера содержит ссылки на то, что можно сделать дальше (в англоязычных источниках это называют HATEOAS — Hypermedia As The Engine Of Application State). Простой пример — обычная ссылка в HTML: она сообщает клиенту, какой запрос сделать дальше.

Основные методы HTTP

Идемпотентность — свойство операции: сколько раз её ни повторишь с одними и теми же данными, результат на сервере будет одинаковым (как умножение на единицу: $x = x \times 1 = (x \times 1) \times 1 = \dots$).

Метод	Назначение	Безопасный	Идемпотентный
GET	Получить представление ресурса	Да	Да
POST	Создать новый ресурс	Нет	Нет
PUT	Полностью заменить ресурс	Нет	Да
PATCH	Частично изменить ресурс	Нет	Нет
DELETE	Удалить ресурс	Нет	Да

«Безопасный» метод не должен менять состояние сервера (только чтение). «Идемпотентный» — повторный вызов с теми же данными не меняет результат: повторный DELETE того же ресурса просто ничего не находит, а повторный PUT перезаписывает ресурс тем же значением. У POST нет ни одного из этих свойств — каждый вызов создаёт новый ресурс.

Коды ответа HTTP

Диапазон	Группа	Примеры
100–199	Информационные	100 Continue
200–299	Успешные	200 OK, 201 Created, 204 No Content
300–399	Перенаправления	301 Moved Permanently, 307 Temporary Redirect
400–499	Ошибки клиента	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
500–599	Ошибки сервера	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Правила формирования URL для REST API

Хороший API интуитивно понятен по одним только адресам (их называют **эндпоинтами**). Основные правила:

- Существительные во множественном числе: /users, /contacts — а не глаголы или единственное число.
- Конкретные названия вместо абстрактных: /videos, а не /items.
- Сложную логику (фильтры, пагинация, сортировка) прячут за query-параметрами после ?, а не в самом пути.
- Действие определяется HTTP-методом, а не словом в адресе.

Плохо	Почему
/api/users/13/remove	Глагол в пути — действие уже задаёт метод DELETE
/api/getusers	Глагол вместо существительного
/api/v1/users-get	То же самое, ещё и с версией в неправильном месте

Задача	Метод	URL
Создать клиента	POST	/customers
Получить/изменить/удалить клиента с id 112233	GET / PUT / DELETE	/customers/112233
Список заказов клиента с id 332244	GET	/customers/332244/orders
Пагинация	GET	/resources?offset=0&limit=25
Фильтрация	GET	/users?filter="name::sam city::denver"
Сортировка (по убыванию — префикс «-»)	GET	/users?sort=lastName firstName birthdate

Базовый пример REST API

Реализуем CRUD (Create, Read, Update, Delete — создание, чтение, обновление, удаление) для абстрактной сущности `item`. Маршрут — существительное во множественном числе `/items`, конкретная запись адресуется через `/items/{item_id}`:

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Item(BaseModel):
    name: str
    description: str = None
    price: float
    tax: float = None

@app.post("/items")
async def create_item(item: Item):
    return item

@app.get("/items/{item_id}")
async def read_item(item_id: int):
    return {"item_id": item_id}

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    return {"item_id": item_id, "item": item}

@app.delete("/items/{item_id}")
async def delete_item(item_id: int):
    return {"item_id": item_id}
```

`POST /items` создаёт элемент из тела запроса, провалидированного моделью `Item`. `GET /items/{item_id}` и `DELETE /items/{item_id}` работают по `id` из пути. `PUT /items/{item_id}` принимает `id` из пути, и новое представление ресурса из тела запроса — так задаётся полное обновление. Ровно эти четыре операции и есть минимальный REST API для одной сущности.

Архитектура приложения: практика

Дальше строим полноценный REST API — сервис заметок (notes) с тегами (tags) — с настоящей многоуровневой архитектурой, базой данных и миграциями.

Многоуровневая архитектура

Подход разделяет систему на независимые уровни, каждый со своей зоной ответственности:

Уровень	Отвечает за
Presentation (API)	Приём HTTP-запросов и формирование ответов для клиента
Business logic (Service)	Правила и алгоритмы работы приложения, координация между уровнями
Data access (Repository)	Непосредственная работа с базой данных

Уровни взаимодействуют только через чёткие интерфейсы — API обращается к сервису, сервис к репозиторию. Это упрощает структуру кода и поддержку: изменения в одном слое почти не затрагивают остальные. Минус — для маленьких проектов такая структура может быть избыточной и добавлять лишние абстракции.

```

# Data access layer (DAL)
class UserRepository:
    def get_user(self, user_id):
        return {"id": user_id, "name": "John Doe"}

# Business logic layer (BLL)
class UserService:
    def __init__(self, user_repository):
        self.user_repository = user_repository

    def get_user_details(self, user_id):
        user = self.user_repository.get_user(user_id)
        return f"User {user['name']} (ID: {user['id']})"

# Presentation layer (UI)
class UserController:
    def __init__(self, user_service):
        self.user_service = user_service

    def display_user(self, user_id):
        print(self.user_service.get_user_details(user_id))

```

Каждый класс отвечает ровно за одну вещь: `UserRepository` — за данные, `UserService` — за логику, `UserController` — за взаимодействие с пользователем. Это уже соответствует части принципов SOLID — единой ответственности и инверсии зависимостей.

Структура проекта

```

├─ src
│   ├── api
│   │   ├── notes.py
│   │   └── tags.py
│   ├── services
│   │   ├── notes.py
│   │   └── tags.py
│   ├── repository
│   │   ├── notes.py
│   │   └── tags.py
│   ├── database
│   │   ├── models.py
│   │   └── db.py
│   ├── conf
│   │   └── config.py
│   └── schemas.py
├─ pyproject.toml
└─ main.py

```

Путь	Роль
src/api/	Presentation-уровень: маршруты (эндпоинты) FastAPI
src/services/	Уровень бизнес-логики, координирует API и репозитории
src/repository/	Уровень доступа к данным, работа с базой
src/database/models.py	ORM-модели таблиц
src/database/db.py	Подключение к базе данных
src/schemas.py	Pydantic-модели для валидации входа/выхода
src/conf/config.py	Конфигурация проекта
main.py	Точка входа в приложение

Подключение к базе данных (асинхронное)

```
class Config:
    DB_URL = "postgresql+asyncpg://postgres:567234@localhost:5432/todo_app"

config = Config
```

Строка подключения — `postgresql` (тип БД) + `asyncpg` (асинхронный драйвер) + логин/пароль + хост/порт + имя базы (`todo_app`). Драйвер ставится отдельно: `poetry add asyncpg`. Сама база поднимается тем же Docker-контейнером, что и раньше: `docker run --name some-postgres -p 5432:5432 -e POSTGRES_PASSWORD=567234 -d postgres`.

```
import contextlib
from sqlalchemy.exc import SQLAlchemyError
from sqlalchemy.ext.asyncio import AsyncEngine, async_sessionmaker, create_async_engine

class DatabaseSessionManager:
    def __init__(self, url: str):
        self._engine: AsyncEngine | None = create_async_engine(url)
        self._session_maker = async_sessionmaker(autoflush=False, autocommit=False, bind=self._engine)

    @contextlib.asynccontextmanager
    async def session(self):
        session = self._session_maker()
        try:
            yield session
        except SQLAlchemyError:
            await session.rollback()
            raise
        finally:
            await session.close()

sessionmanager = DatabaseSessionManager(config.DB_URL)

async def get_db():
    async with sessionmanager.session() as session:
        yield session
```

`DatabaseSessionManager` создаёт асинхронный engine и фабрику сессий. Метод `session()` — асинхронный контекстный менеджер: открывает сессию, при ошибке SQLAlchemy откатывает транзакцию (`rollback`), в любом случае закрывает сессию в конце. `get_db` — та же самая знакомая зависимость для `Depends()`, только теперь асинхронная.

`autocommit=False`, `autoflush=False` отключают автоматическое сохранение изменений — это даёт полный контроль над транзакцией: можно выполнить несколько операций и явно решить, зафиксировать их разом (`commit`) или откатить (`rollback`) при ошибке.

Модели (ORM)

```
class Base(DeclarativeBase):
    pass

note_m2m_tag = Table(
    "note_m2m_tag", Base.metadata,
    Column("note_id", Integer, ForeignKey("notes.id", ondelete="CASCADE")),
    Column("tag_id", Integer, ForeignKey("tags.id", ondelete="CASCADE")),
    PrimaryKeyConstraint("note_id", "tag_id"),
)

class Note(Base):
    __tablename__ = "notes"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(50), nullable=False)
    created_at: Mapped[datetime] = mapped_column(default=func.now())
    updated_at: Mapped[datetime] = mapped_column(default=func.now(), onupdate=func.now())
    description: Mapped[str] = mapped_column(String(150), nullable=False)
    done: Mapped[bool] = mapped_column(default=False)
    tags: Mapped[list["Tag"]] = relationship("Tag", secondary=note_m2m_tag, backref="notes")

class Tag(Base):
    __tablename__ = "tags"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(25), nullable=False, unique=True)
```

Связь «многие ко многим» между заметками и тегами хранится в отдельной таблице `note_m2m_tag` — по строке на каждую пару (note, tag). `ondelete="CASCADE"` означает: удалили заметку или тег — автоматически удалятся и связанные строки в этой таблице.

В `created_at` / `updated_at` — `func.now()` подставляет текущее время при создании, а `onupdate=func.now()` — обновляет его при каждом изменении записи, без ручного кода.

Связь `tags = relationship("Tag", secondary=note_m2m_tag, backref="notes")`: `secondary` указывает промежуточную таблицу, а `backref="notes"` добавляет в класс `Tag` обратное свойство `notes` — то есть у тега тоже появляется доступ к списку его заметок, без дублирования кода.

Миграции (Alembic)

Alembic — инструмент для управления изменениями структуры базы данных (миграциями), синхронизированный с моделями SQLAlchemy.

```
alembic init migrations
```

В сгенерированном файле `migrations/env.py` нужно вручную указать две вещи — модели, по которым генерировать миграции, и актуальную строку подключения:

```
from src.database.models import Base
from src.conf.config import config as app_config

target_metadata = Base.metadata
config.set_main_option("sqlalchemy.url", app_config.DB_URL)
```

Поскольку приложение работает асинхронно (драйвер `asyncpg`), функцию запуска миграций в `env.py` тоже нужно переписать на асинхронный движок (`async_engine_from_config`) — это готовый шаблонный код, который просто копируется в проект без необходимости заучивать наизусть.

```
alembic revision --autogenerate -m 'Init'
alembic upgrade head
```

Первая команда сравнивает модели с текущим состоянием базы и генерирует файл миграции; вторая — применяет миграцию, создавая реальные таблицы в базе.

Pydantic-схемы

```
class TagModel(BaseModel):
    name: str = Field(max_length=25)

class TagResponse(TagModel):
    id: int
    model_config = ConfigDict(from_attributes=True)

class NoteBase(BaseModel):
    title: str = Field(max_length=50)
    description: str = Field(max_length=150)

class NoteModel(NoteBase):
    tags: List[int]

class NoteUpdate(NoteModel):
    done: bool

class NoteStatusUpdate(BaseModel):
    done: bool

class NoteResponse(NoteBase):
    id: int
    done: bool
    created_at: datetime | None
    updated_at: Optional[datetime] | None
    tags: List[TagResponse] | None
    model_config = ConfigDict(from_attributes=True)
```

Для каждой сущности — отдельные модели под разные задачи: `*Model` / `*Base` — для входных данных, `*Response` — для ответа клиенту, `NoteUpdate` — для полного обновления, `NoteStatusUpdate` — только для смены статуса `done`.

`model_config = ConfigDict(from_attributes=True)` позволяет Pydantic строить модель не только из словаря, но и из атрибутов обычного объекта — то есть напрямую из ORM-объекта SQLAlchemy, без ручного преобразования каждого поля.

CRUD-операции для тегов

Действие	Метод	URL
Список тегов	GET	/api/tags
Создать тег	POST	/api/tags
Тег по id	GET	/api/tags/{id}
Обновить тег	PUT	/api/tags/{id}
Удалить тег	DELETE	/api/tags/{id}

Presentation: `src/api/tags.py` — маршруты через `APIRouter`, вся логика делегирована сервису:

```
from fastapi import APIRouter, HTTPException, Depends, status
from sqlalchemy.ext.asyncio import AsyncSession
from src.database.db import get_db
from src.schemas import TagModel, TagResponse
from src.services.tags import TagService

router = APIRouter(prefix="/tags", tags=["tags"])

@router.get("/", response_model=List[TagResponse])
async def read_tags(skip: int = 0, limit: int = 100, db: AsyncSession = Depends(get_db)):
    return await TagService(db).get_tags(skip, limit)

@router.get("/{tag_id}", response_model=TagResponse)
async def read_tag(tag_id: int, db: AsyncSession = Depends(get_db)):
    tag = await TagService(db).get_tag(tag_id)
    if tag is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Tag not found")
    return tag

@router.post("/", response_model=TagResponse, status_code=status.HTTP_201_CREATED)
async def create_tag(body: TagModel, db: AsyncSession = Depends(get_db)):
    return await TagService(db).create_tag(body)

@router.put("/{tag_id}", response_model=TagResponse)
async def update_tag(body: TagModel, tag_id: int, db: AsyncSession = Depends(get_db)):
    tag = await TagService(db).update_tag(tag_id, body)
    if tag is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Tag not found")
    return tag

@router.delete("/{tag_id}", response_model=TagResponse)
async def remove_tag(tag_id: int, db: AsyncSession = Depends(get_db)):
    tag = await TagService(db).remove_tag(tag_id)
    if tag is None:
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail="Tag not found")
    return tag
```

Service: `src/services/tags.py` — пока просто передаёт вызовы репозиторию (сюда позже добавляется бизнес-логика, если появится):

```
class TagService:
    def __init__(self, db: AsyncSession):
        self.repository = TagRepository(db)

    async def create_tag(self, body): return await self.repository.create_tag(body)
    async def get_tags(self, skip, limit): return await self.repository.get_tags(skip, limit)
    async def get_tag(self, tag_id): return await self.repository.get_tag_by_id(tag_id)
    async def update_tag(self, tag_id, body): return await self.repository.update_tag(tag_id, body)
    async def remove_tag(self, tag_id): return await self.repository.remove_tag(tag_id)
```

Repository: `src/repository/tags.py` — единственный слой, который реально трогает базу:

```
class TagRepository:
    def __init__(self, session: AsyncSession):
        self.db = session

    async def get_tags(self, skip, limit):
        stmt = select(Tag).offset(skip).limit(limit)
        result = await self.db.execute(stmt)
        return result.scalars().all()

    async def get_tag_by_id(self, tag_id):
        stmt = select(Tag).filter_by(id=tag_id)
        result = await self.db.execute(stmt)
        return result.scalar_one_or_none()

    async def create_tag(self, body):
        tag = Tag(**body.model_dump(exclude_unset=True))
        self.db.add(tag)
        await self.db.commit()
        await self.db.refresh(tag)
        return tag

    async def update_tag(self, tag_id, body):
        tag = await self.get_tag_by_id(tag_id)
        if tag:
            tag.name = body.name
            await self.db.commit()
            await self.db.refresh(tag)
        return tag

    async def remove_tag(self, tag_id):
        tag = await self.get_tag_by_id(tag_id)
        if tag:
            await self.db.delete(tag)
            await self.db.commit()
        return tag
```

`body.model_dump(exclude_unset=True)` превращает Pydantic-модель в словарь, включая только реально переданные поля — это и позволяет делать частичные обновления, не затирая остальные поля значениями по умолчанию.

CRUD-операции для заметок

Устроено аналогично тегам, с двумя важными отличиями: у заметки есть связь с тегами, а обновление статуса выделено в отдельный маршрут.

Действие	Метод	URL
Список заметок	GET	/api/notes
Создать заметку	POST	/api/notes
Заметка по id	GET	/api/notes/{id}
Полностью обновить заметку	PUT	/api/notes/{id}
Обновить только статус	PATCH	/api/notes/{id}
Удалить заметку	DELETE	/api/notes/{id}

Почему PUT для полного обновления и PATCH для статуса? PUT подразумевает замену ресурса целиком, PATCH — частичное изменение. Когда меняется только поле `done`, семантически правильнее PATCH: он явно говорит «меняю одно свойство», а не «присылаю новую версию всего объекта».

Маршруты (`src/api/notes.py`) написаны по тому же шаблону, что и для тегов — `APIRouter` , `Depends(get_db)` , `response_model` , проверка на `404` . Интересное — на уровне сервиса и репозитория.

```
# src/services/notes.py
class NoteService:
    def __init__(self, db: AsyncSession):
        self.note_repository = NoteRepository(db)
        self.tag_repository = TagRepository(db)

    async def create_note(self, body):
        tags = await self.tag_repository.get_tags_by_ids(body.tags)
        return await self.note_repository.create_note(body, tags)

    async def update_note(self, note_id, body):
        tags = await self.tag_repository.get_tags_by_ids(body.tags)
        return await self.note_repository.update_note(note_id, body, tags)

    async def update_status_note(self, note_id, body):
        return await self.note_repository.update_status_note(note_id, body)
```

`NoteService` работает сразу с двумя репозиториями: перед созданием или обновлением заметки он подтягивает объекты `Tag` по переданным `id` из `tag_repository` , а затем передаёт их в `note_repository` — координация нескольких источников данных как раз и есть работа сервисного слоя.

```
# src/repository/notes.py
class NoteRepository:
    def __init__(self, session: AsyncSession):
        self.db = session

    async def get_notes(self, skip, limit):
        stmt = select(Note).options(selectinload(Note.tags)).offset(skip).limit(limit)
        result = await self.db.execute(stmt)
        return result.scalars().all()

    async def create_note(self, body, tags):
        note = Note(**body.model_dump(exclude={"tags"}, exclude_unset=True), tags=tags)
        self.db.add(note)
        await self.db.commit()
        await self.db.refresh(note)
        return await self.get_note_by_id(note.id)

    async def update_status_note(self, note_id, body):
        note = await self.get_note_by_id(note_id)
        if note:
            note.done = body.done
            await self.db.commit()
            await self.db.refresh(note)
        return note
```

Проблема N+1. Если для каждой заметки отдельно запрашивать её теги, на 100 заметок получится 101 запрос к базе (1 + по одному на каждую). `selectinload(Note.tags)` подгружает все теги для всей выборки заметок одним дополнительным запросом — вместо сотен, всего два запроса в сумме.

Метод `create_note` использует `exclude={"tags"}` , потому что теги — не обычное поле, а связь, которая устанавливается отдельно через параметр `tags=tags` , а не через распаковку словаря.

Сборка приложения

Отдельный healthcheck-роутер (`src/api/utils.py`) — версия из более ранней темы, только теперь с асинхронным запросом `await db.execute(text("SELECT 1"))` . В `main.py` все роутеры подключаются к приложению через

`include_router` с общим префиксом:

```
from fastapi import FastAPI
from src.api import tags, utils, notes

app = FastAPI()
app.include_router(utils.router, prefix="/api")
app.include_router(tags.router, prefix="/api")
app.include_router(notes.router, prefix="/api")
```

`prefix="/api"` добавляется ко всем маршрутам модуля разом — так `/tags` из роутера превращается в реальный `/api/tags`.

Готовое API тестируется через Swagger (`/docs`) или инструменты вроде Postman — до подключения настоящего клиента (React/Vue/Angular). Если создать заметку без обязательного поля `description`, сервер вернёт `422 Unprocessable Entity` с указанием, какое именно поле отсутствует — Pydantic-валидация отрабатывает автоматически, без единой ручной проверки.